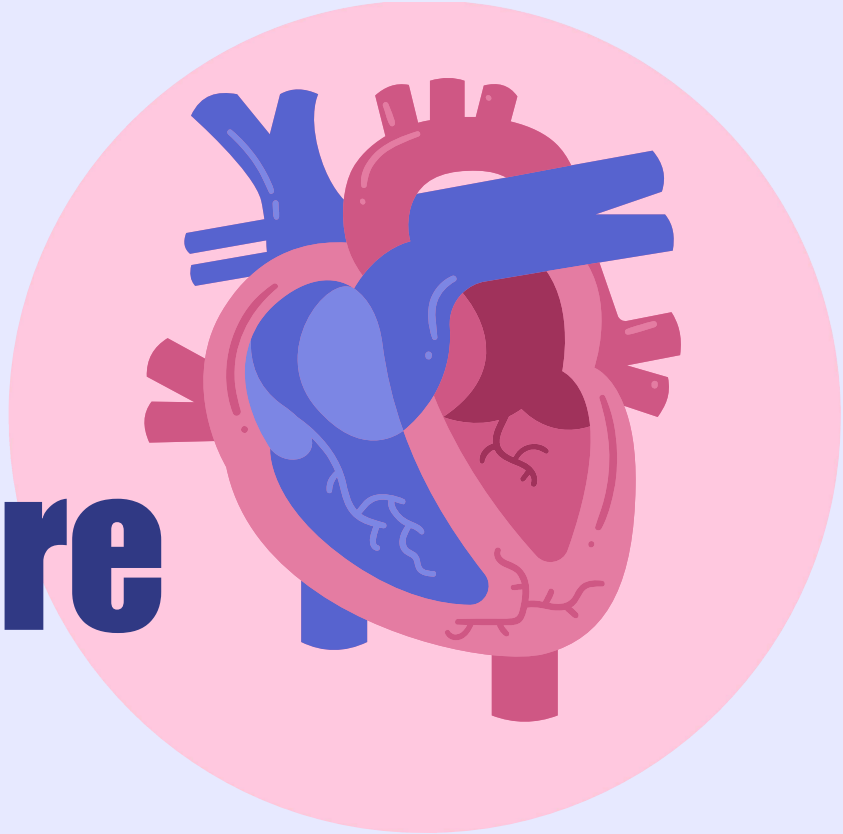
Machine Learning Project

Heart Failure Prediction



A project submitted for the Data Analysis 1 course (DS2313)

Introduction to Heart Failure Prediction

This project focuses on developing a machine learning model to predict heart failure using clinical patient data. Heart disease is one of the leading causes of death worldwide, making early detection essential for improving patient outcomes. The study analyzes the Heart Failure Prediction Dataset to understand the key clinical factors associated with heart disease and to evaluate the performance of different classification models.

This research aims to answer the following questions:

1. Which clinical features are most strongly associated with heart failure?
2. Can machine learning models accurately predict whether a patient has heart disease?
3. Which model provides the best performance for this classification task?

To address these questions, the dataset is explored, cleaned, and processed to build and compare machine learning models that support medical decision-making.

Section 1.

Data Load & inspection

```
تحميل البيانات من المسار الصح / Load dataset from correct path
= pd.read_csv("/kaggle/input/datasets/fedesoriano/heart-failure-predic
heart.csv")
```

```
# عرض أول 5 صفوف / Preview first 5 rows
df.head()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
# عرض عدد الصفوف والأعمدة / Show shape
df.shape
```

```
918, 12)
```

Section 1. Data Load & inspection

```
# عرض معلومات البيانات / Show info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
```

```
# عرض الإحصائيات / Show statistics
df.describe()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

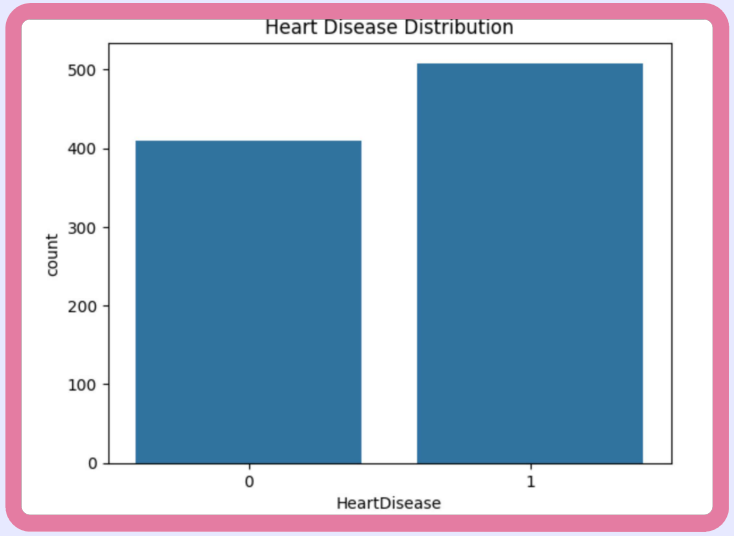
Section 2: Exploratory Data Analysis & Visualization (EDA)

Heart Disease Distribution

```
# عرض توزيع مرض القلب / Visualize heart disease distribution
sns.countplot(x='HeartDisease', data=df)

plt.title('Heart Disease Distribution')

plt.show()
```



Section 2: Exploratory Data Analysis & Visualization (EDA)

Age distribution

```
# توزيع الأعمار / Age distribution
plt.figure(figsize=(8,5))

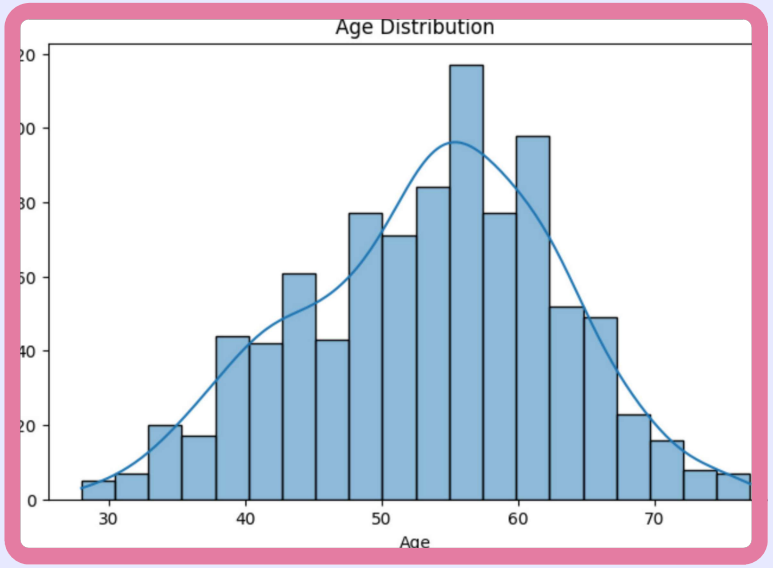
sns.histplot(df['Age'], bins=20, kde=True)

plt.title('Age Distribution')

plt.xlabel('Age')

plt.ylabel('Count')

plt.show()
```



Section 2: Exploratory Data Analysis & Visualization (EDA)

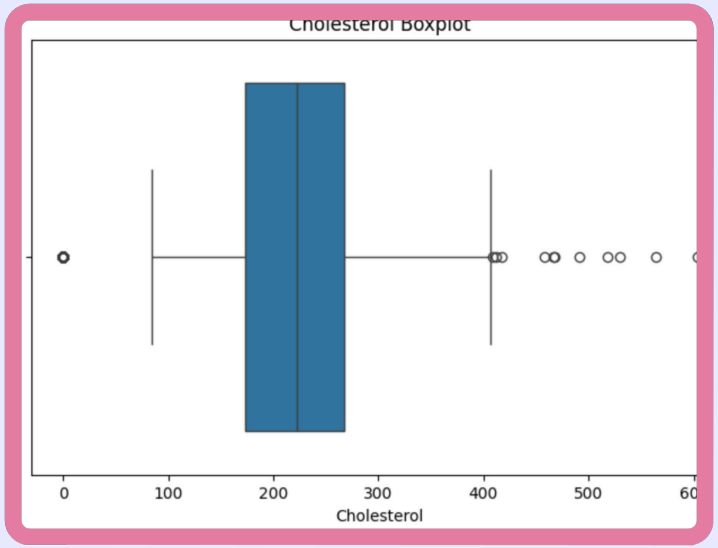
Cholesterol distribution and outliers

```
# توزيع الكوليسترول واكتشاف القيم الشاذة / Cholesterol distribution and outliers
plt.figure(figsize=(8,5))

sns.boxplot(x=df['Cholesterol'])

plt.title('Cholesterol Boxplot')

plt.show()
```



Section 2: Exploratory Data Analysis & Visualization (EDA)

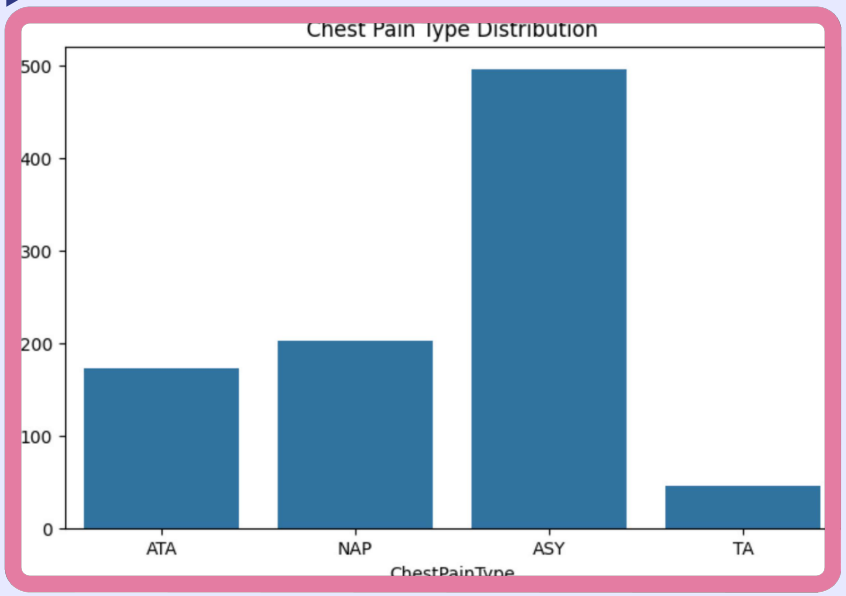
Chest pain type distribution

```
# توزيع أنواع ألم الصدر / Chest pain type distribution
plt.figure(figsize=(8,5))

sns.countplot(x='ChestPainType', data=df)

plt.title('Chest Pain Type Distribution')

plt.show()
```



Section 2: Exploratory Data Analysis & Visualization (EDA)

Correlation heatmap

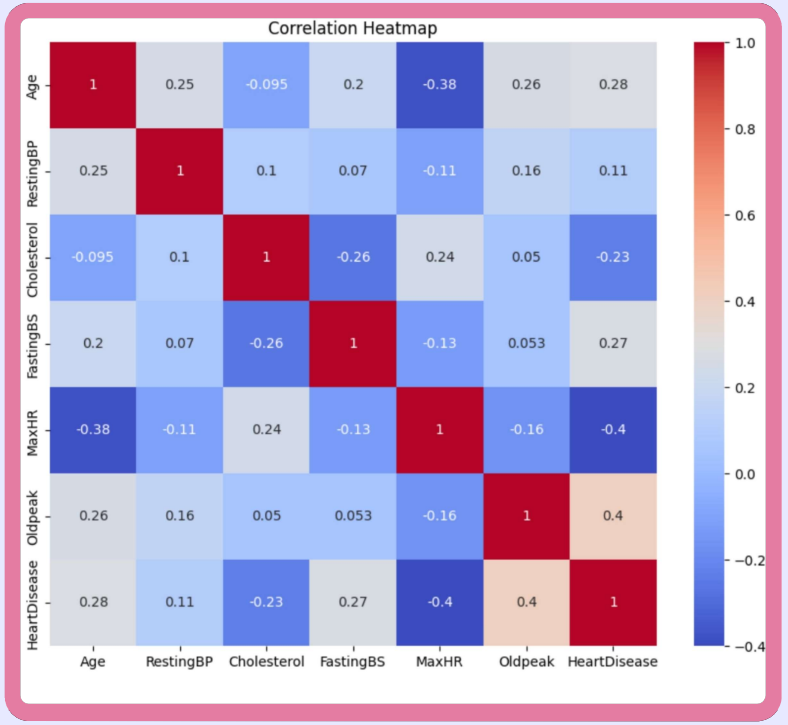
```
# خريطة الارتباط بين المتغيرات / Correlation heatmap
plt.figure(figsize=(10,8))

numeric_df = df.select_dtypes(include=['int64', 'float64']

sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm'

plt.title('Correlation Heatmap')

plt.show()
```



Section 3: Data Cleaning and Preprocessing

Checking for Missing Values:

```
print("Missing Values: ")
print(df.isnull().sum())
```

```
Missing Values:
Age               0
Sex               0
ChestPainType     0
RestingBP         0
Cholesterol       0
FastingBS         0
RestingECG        0
MaxHR             0
ExerciseAngina    0
Oldpeak           0
ST_Slope          0
HeartDisease      0
```

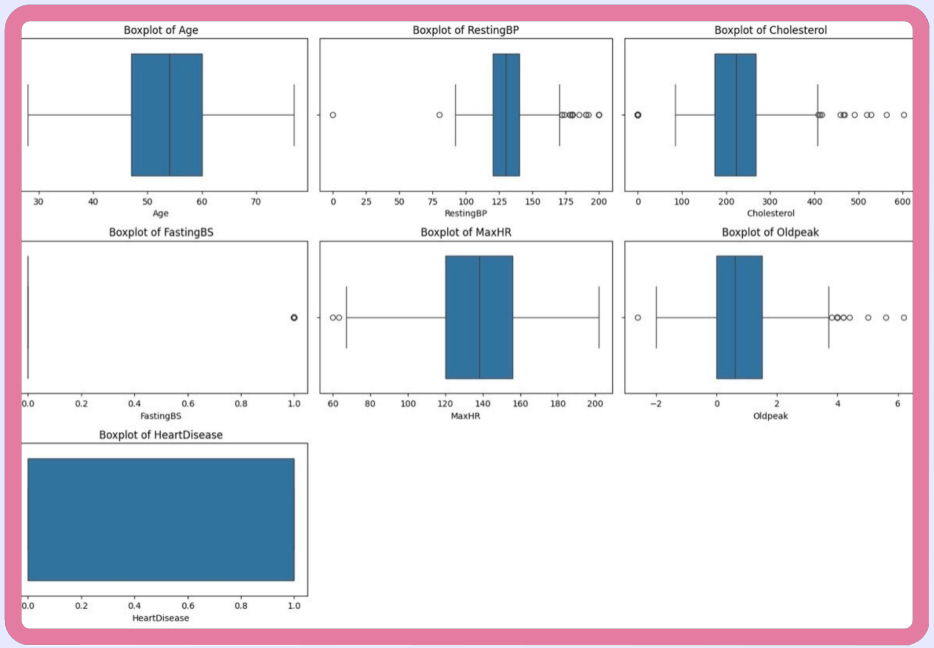
Section 3: Data Cleaning and Preprocessing

Checking and Handling Outliers:

```
for i, col in enumerate(numeric_df, 1):
    plt.subplot(3, 3, i)

    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col}')

plt.tight_layout()
plt.show()
```



Section 3: Data Cleaning and Preprocessing

Checking and Handling Outliers:

```
    # تقييد (تسقيف) القيم الشاذة
    df[column] = df[column].clip(lower=low
er, upper=upper)
    return df

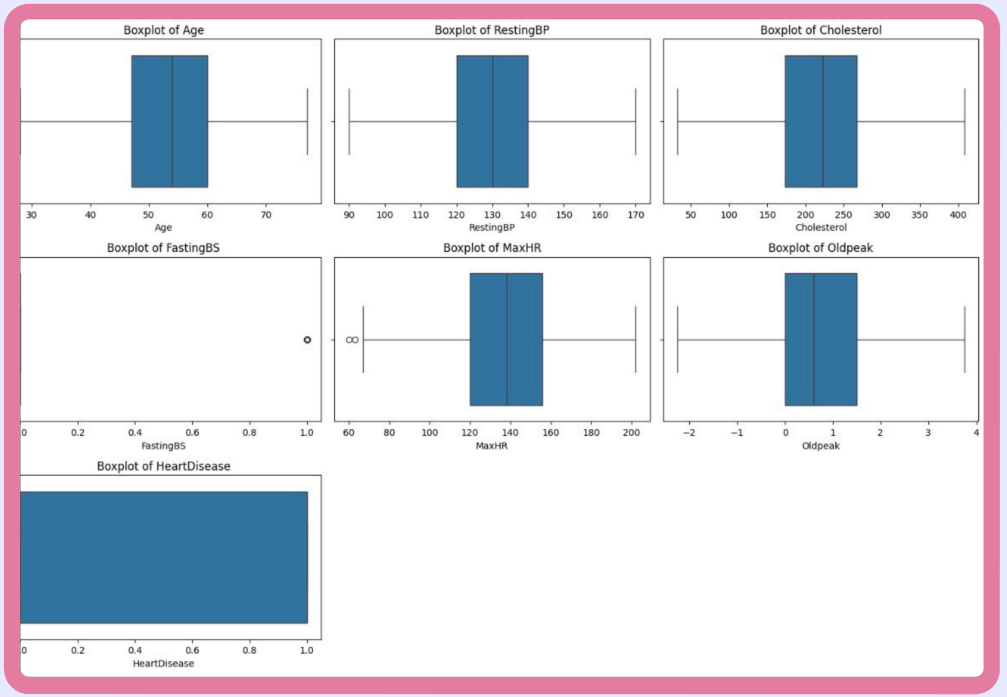
# الأعمدة التي تحتوي على قيم شاذة
cols = ['RestingBP', 'Cholesterol', 'Oldpe
ak']

# تطبيق معالجة القيم الشاذة على كل عمود
for col in cols:
    df = cap_outliers(df, col)

# الرسومات بعد معالجة القيم الشاذة
)lt.figure(figsize=(15, 10))
 or i, col in enumerate(numeric_df, 1):
    plt.subplot(3, 3, i)

    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col}')

plt.tight_layout()
plt.show()
```



Section 3: Data Cleaning and Preprocessing

Checking Duplicate Rows:

```
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows : ", dupli
cate_count,)


Number of duplicate rows :  0
```

Section 3: Data Cleaning and Preprocessing

Checking Categorical Variables using unique():

```
#تحديد الأعمدة النصية في البيانات
categorical_cols = df.select_dtypes(includ
e=['object']).columns

print("Unique values in categorical column
s :")
for col in categorical_cols:
    print("Column '", col, "': ", df[col].
unique())
```

```
Unique values in categorical columns :
Column ' Sex ':  ['M' 'F']
Column ' ChestPainType ':  ['ATA' 'NAP' 'A
SY' 'TA']
Column ' RestingECG ':  ['Normal' 'ST' 'LV
H']
Column ' ExerciseAngina ':  ['N' 'Y']
Column ' ST_Slope ':  ['Up' 'Flat' 'Down']
```

Section 4: Data Preprocessing & Training Preparation

One-Hot Encoding :

```
# تحويل المتغيرات الفئوية إلى رقمية باستخدام One-Hot Encoding
# نستخدم drop_first=True لتجنب مشكلة الـ Multicollinearity
df_encoded = pd.get_dummies(df, drop_first=True)

# عرض أول 5 صفوف للتأكد من نجاح العملية
df_encoded.head()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease	Sex_M	ChestPainType_ATA	ChestPainType_NAP
0	40	140	289.0	0	172	0.0	0	True	True	False
1	49	160	180.0	0	156	1.0	1	False	False	True
2	37	130	283.0	0	98	0.0	0	True	True	False
3	48	138	214.0	0	108	1.5	1	False	False	False
4	54	150	195.0	0	122	0.0	0	True	False	True

Section 4: Data Preprocessing & Training Preparation

Splitting Data :

```python
# تحديد المتغيرات المستقلة (X) والمتغير التابع (y)
X = df_encoded.drop('HeartDisease', axis=1)
y = df_encoded['HeartDisease']

# تقسيم البيانات إلى تدريب واختبار (80% تدريب، 20% اختبار)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# التأكد من تقسيم البيانات بشكل صحيح
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

```
Training set shape: (734, 15)
Testing set shape: (184, 15)
```

Section 4: Data Preprocessing & Training Preparation

Feature Scaling :

```
# تهيئة الـ scaler
scaler = StandardScaler()

# تطبيق التحجيم على بيانات التدريب والاختبار
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# عرض عينة من البيانات بعد التحجيم
print("تم تحجيم البيانات بنجاح.")
```

```
تم تحجيم البيانات بنجاح.
```

Section 5:

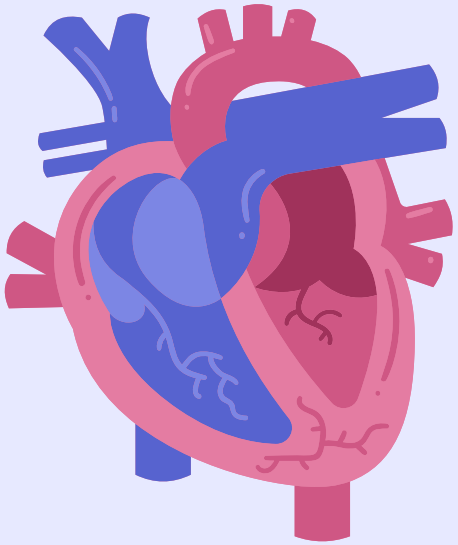
Training and Prediction



1. I trained two models: Logistic Regression and Decision Tree Classifier to compare their performance on the heart disease dataset. Loops were used to find the best parameters — max_iter = 1100 and max_depth = 4.

2. After training and prediction, results showed both models performed similarly, but the Decision Tree outperformed in all metrics.

#5.1 RESULTS AND DISCUSSION



Both models were evaluated using four metrics: Accuracy, Precision, Recall, and F1-score. Results showed similar performance, but the Decision Tree outperformed in all metrics.

The Decision Tree achieved a higher Recall (0.879 vs 0.841), which is crucial in medical applications. High Recall means the model detects ill patients more accurately — vital for heart disease prediction

Overall, the Decision Tree was the better model for this dataset, as it delivered stronger and more reliable results than Logistic Regression.

Conclusion

Objective: Built a classification model to predict heart disease using clinical data.

Key Findings: The Decision Tree model outperformed Logistic Regression, proving highly effective for this task.

Experience: Successfully executed the full machine learning cycle: data preprocessing, model training, and evaluation.

Future Work: We aim to improve performance by testing advanced algorithms like Random Forest and using larger, more diverse datasets.



Thank You